

SPECYFIKACJA PROJEKTU

=====

Temat:

Automatyzacja obsługi plików Excel z wykorzystaniem języka Python

Cel główny:

Stworzenie narzędzia (skryptu lub aplikacji) w Pythonie, które:

- Pobiera dane z co najmniej dwóch plików Excel.
- Dokonuje złożonych operacji na danych (scalanie, walidacja, obliczenia).
- Generuje końcowy raport lub plik wynikowy.

Przykładowy scenariusz:

1) Mamy plik bazowy (np. dane_główne.xlsx) zawierający informacje o pracownikach, produktach lub dowolnej innej głównej encji.

2) Mamy dodatkowe pliki Excel (np. czesc_danych_01.xlsx, czesc_danych_02.xlsx), w których występują dane szczegółowe (np. liczba przepracowanych godzin, zamówienia, sprzedaż itp.).

3) Skrypt:

- Wczytuje dane z pliku bazowego oraz wszystkich plików dodatkowych.
- Waliduje poprawność (np. czy ID istnieje, czy wartości godzin są liczbami, czy kolumny mają właściwe nazwy).
- Łączy (merge) dane w jedną strukturę na podstawie kolumny kluczowej (np. ID_pracownika).
- Dokonuje obliczeń (np. sumuje godziny, mnoży przez stawkę godzinową, wylicza prowizje).
- Generuje jeden plik Excel raportowy (np. raport_koncowy.xlsx) zawierający:
 - a) Arkusz ze szczegółowymi danymi (zsumowane dane z wielu plików).
 - b) Arkusz z podsumowaniami (np. łączna liczba godzin, łączny koszt, podział według działów).

Proponowany zakres funkcjonalności:

1) Wczytywanie i walidacja danych

- Czytanie plików Excel (pandas.read_excel / openpyxl).
- Sprawdzanie obecności wymaganych kolumn (np. 'ID', 'Godziny', 'Data').
- Obsługa wielu plików jednocześnie (np. pętla po folderze z plikami).
- Informowanie o błędach i zapisywanie nieprawidłowych rekordów do osobnego pliku (opcjonalnie).

2) Scalanie i obróbka

- Scalanie danych na podstawie klucza (np. ID_pracownika) z plikiem bazowym.
- Grupowanie i sumowanie (groupby), np. total godzin dla każdego pracownika.
- Dodatkowe obliczenia: wynagrodzenie, prowizje, koszty.

3) Generowanie raportu

- Zapis do nowego pliku Excel z wieloma arkuszami (dane szczegółowe i zestawienie).
- Formatowanie (np. nagłówki, szerokość kolumn, format walutowy).
- (Opcjonalnie) Dodanie wykresu (matplotlib lub xlsxwriter).

4) Funkcje dodatkowe (opcjonalnie)

- Generowanie raportu PDF (reportlab, pdfkit, itp.).
- Wysyłka e-mail z raportem w załączniku.
- Harmonogram automatycznego uruchamiania (cron lub Task Scheduler).
- Podstawowe testy jednostkowe (unittest/pytest) sprawdzające poprawność agregacji.

Przykładowa struktura oddawanych materiałów:

-
- 1) Pliki .xlsx z danymi przykładowymi (np. dane_główne.xlsx + 2-3 pliki z częściami danych).
 - 2) Skrypt Python (main.py / raport_generator.py) z komentarzami.
 - 3) Plik README.md z opisem:
 - Jak uruchomić projekt (jakie biblioteki zainstalować).
 - Struktura kolumn w plikach .xlsx.
 - Przykład użycia i oczekiwany wynik.
 - 4) (Opcjonalnie) Testy jednostkowe i/lub krótka prezentacja (2-3 slajdy).

Kryteria oceny:

-
- 1) Poprawność implementacji (czy raport jest zgodny z założeniami, czy dane są prawidłowo scalone i obliczone).
 - 2) Jakość kodu (czytelność, podział na funkcje/klasy, zgodność z PEP 8).
 - 3) Zakres funkcjonalności (podstawowy vs. dodatkowe elementy).
 - 4) Dokumentacja (README, opis kolumn, przykładowe wywołania).
 - 5) Przygotowanie danych przykładowych i sposób ich prezentacji.

Podsumowanie:

-
- Projekt ma na celu zademonstrowanie praktycznych umiejętności związanych z:
- Odczytem i zapisem plików Excel (biblioteki pandas, openpyxl).
 - Przekształcaniem danych (merge, groupby, walidacja).
 - Automatyzacją procesów (generowanie raportów, wysyłanie e-mail, harmonogram).
 - Organizacją kodu i krótką dokumentacją.

Uwaga:

Temat można dostosować do dowolnej dziedziny (np. kadry, sprzedaż, magazyn). Ważne, by ostatecznie dane były scalone i przetworzone w czytelny sposób, a student wykazał się znajomością zarówno języka Python, jak i automatyzacji procesów z udziałem Excela.